Cyba Blockchain Security

Granite

Security Enhancements

Audit Report

May 22, 2026 Version 1.0

Table of Contents

01 | Executive Summary

Granite engaged Cyba Blockchain Security to conduct a security review of the codebase contained within the core-v1 repository.

The engagement was conducted over a period of two days, from 05.05.2026 to 06.05.2026, during which a team of two auditors reviewed the files and commits included in scope.

The audit focused on a manual review of the codebase, with the objective of identifying security vulnerabilities as well as providing codebase enhancement and fortification recommendations. These recommendations were documented as informational findings where applicable.

During the engagement, a total of 7 issues were identified. All identified issues were either resolved by the development team or explicitly acknowledged. In addition, 11 informational findings related to code quality and maintainability were reported, all of which were either resolved or acknowledged by the team.

Overview

Project Name	Security Enhancements
Codebase	https://github.com/GraniteProtocol/core-v1
Operating platform	Stacks
Programming language	Clarity
Initial commit	`cbc7edaf8e7c53c333e56b6549090fc441a3ab9e`
Remediation commit	`c5d35e5bd05f124b8f1f2fd8fb963d365b1da5ec`
Timeline	From 05.05.2026 to 06.05.2026 (2 days)
Audit methodology	Static analysis and manual review
Auditors	Alin Barbatei (ABA), Silverologist

Audit Scope

The audit focused on changes introduced in between two commits:

- From commit: ea0efd7fef4611580ed3d1ab12acb612a7029e35
- To commit: cbc7edaf8e7c53c333e56b6549090fc441a3ab9e
- Exact changes can be seen here

The full change description can be seen in the message for each of the commits that affected the in-scope files: f1e3aee, 6e71adf, 66067ce, 67942fb, e638db1 and cbc7eda.

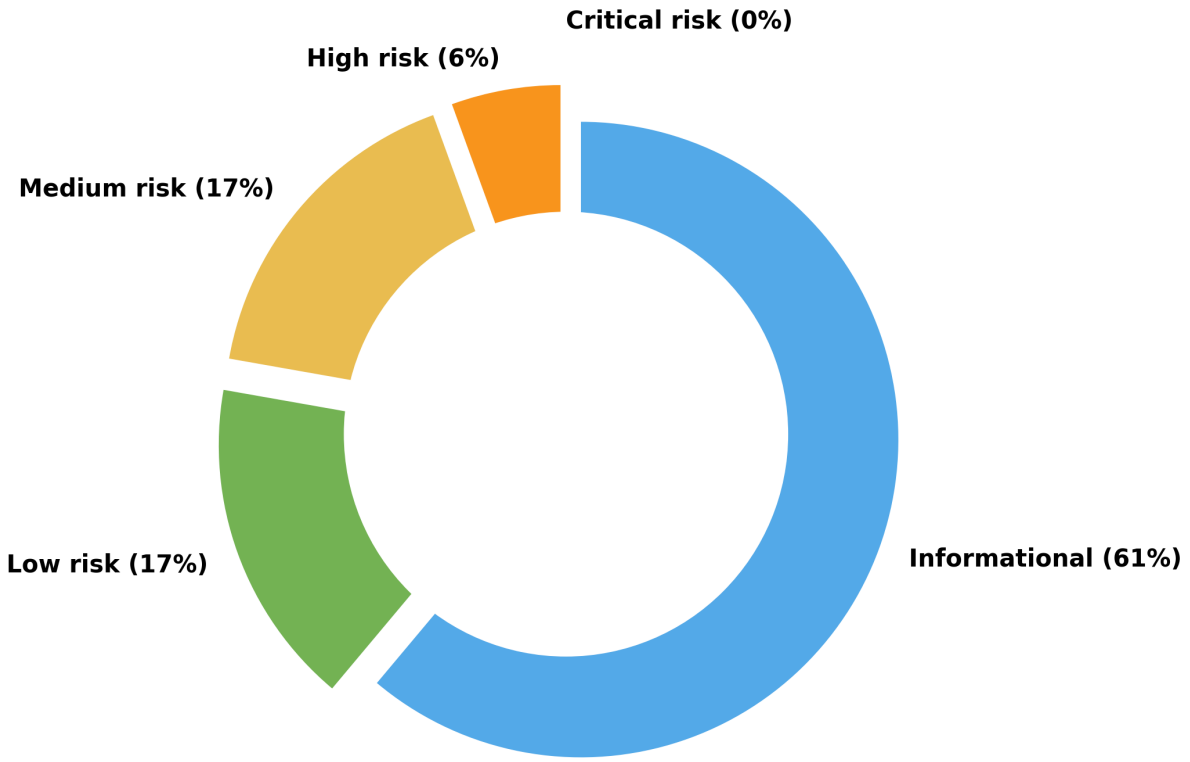
The in-scope files that are touched by the changes are as follows:

Files and folders in scope

- contracts/borrower-v1.clar
- contracts/flash-loan-v1.clar
- contracts/governance-v1.clar
- contracts/liquidator-v1.clar
- contracts/liquidity-provider-v1.clar
- contracts/lp-incentives-v2.clar
- contracts/staking-v1.clar
- contracts/modules/linear-kinked-ir-utility.clar
- contracts/modules/linear-kinked-ir-v1.clar
- contracts/modules/pyth-adapter-v1.clar
- contracts/modules/withdrawal-caps-v1.clar

Summary of Findings

Severity	Total Found	Resolved	Partially Resolved	Acknowledged
Critical risk	0	0	0	0
High risk	1	1	0	0
Medium risk	3	3	0	0
Low risk	3	3	0	0
Informational	11	11	0	0



Findings & Resolutions

ID	Title	Severity	Status
H-01	LPs For New Markets Are Vulnerable to Inflation Attack	High	Resolved
M-01	Interest Dust Assigned to Staking Interest Causes Repay and Liquidation Underflows	Medium	Resolved
M-02	Updating Pyth Token Feeds Has no Timelock Enforcement	Medium	Resolved
M-03	Staking Wipe-Out Detection Gap and Post-Wipe LP Token Leak	Medium	Resolved
L-01	New Depositor Value Extraction After Catastrophic Socialization Zeroes Total Assets	Low	Resolved
L-02	Governance Threshold Calculated at Execution Time With Live Member Count Enables Retroactive Threshold Manipulation	Low	Resolved

ID	Title	Severity	Status
L-03	Disabling Interest Accrual Permanently Skips Pending Borrower Interest	Low	Resolved
I-01	Redundant Reentrancy Guard in Flash Loan Due to Clarity VM Built-in Protection	Informational	Resolved
I-02	Redundant safe-sub in calculate-repayment-info Denominator Computation	Informational	Resolved
I-03	Full Debt Repayment Leaves Phantom Borrowed Amount Residual	Informational	Resolved
I-04	LP Incentives Authorization Check Renders On-Behalf-Of Parameter Ineffective	Informational	Resolved
I-05	Missing Elapsed Time Clamp in Refill Bucket Logic Allows Revert on Timestamp Regression	Informational	Resolved
I-06	Oracle Staleness Minimum of 60 Seconds Is Excessive for Stacks Block Times	Informational	Resolved
I-07	Unbounded Interest Rate Output Combined With Taylor Guard Can Theoretically Block Protocol	Informational	Resolved
I-08	Redundant safe-sub in slash-total-staked-lp-tokens	Informational	Resolved
I-09	Miscellaneous Code Quality Issues	Informational	Resolved
I-10	IR mul Function Uses Different Rounding Between Production and Utility Contracts	Informational	Resolved
I-11	Exponent Upper Bound in Oracle Price Validation Is Too Permissive	Informational	Resolved

02 | Introduction

About Cyba Blockchain Security

Cyba Blockchain Security is a security firm founded by Alin Barbatei (ABA), focused on rigorous smart contract and protocol security across EVM and Bitcoin L2 ecosystems, including Stacks (Clarity).

Built on a foundation of deep Web2 and Web3 security experience, Cyba emphasizes high-quality, adversarial reviews and clear, step-by-step guidance throughout every engagement, helping teams not only identify risks, but understand and resolve them with confidence.

For more information visit cybasecurity.io.

About the Security Enhancements

Granite is a DeFi lending market that offers overcollateralized loans on SIP-10 tokens managed and operated by immutable smart contracts. Granite was created and is managed by Trust Machines, a leading team of engineers, builders and researchers within the Stacks Bitcoin L2 ecosystem.

This engagement reviewed security enhancements applied to the protocol's core contracts following a previous security review. The changes touched borrowing, liquidation, staking, flash loans, governance, LP incentives, interest rate modeling, oracle integration, and withdrawal cap logic. Key areas of focus included:

- **Arithmetic safety** fixes addressing underflow and overflow risks in staking slashes, liquidation accounting, and interest rate accrual
- **Cross-contract** cascade prevention in the liquidation pipeline to stop cascading reverts from propagating across independent operations
- **Staking module** hardening around the wipe-out lifecycle: detection of staking depletion via bad-debt socialization, post-wipe LP token leak prevention, and withdrawal queue finalization under edge conditions
- **Governance contract** fixes for snapshot-based voting thresholds and member count handling across proposal lifecycle operations
- **Oracle adapter** updates, withdrawal cap refinements, and flash loan access control improvements

Issues Risk Classification

The current report contains issues, or findings, that impact the protocol. Depending on the likelihood of the issue appearing and its impact (damage), an issue is in one of four risk categories or severities: *Critical*, *High*, *Medium*, *Low* or *Informational*.

The following table shows an overview of how likelihood and impact determines the severity of an issue.

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost or a functionality of the protocol is affected.
- **Low** - any kind of unexpected behavior that's not so critical.

Likelihood

- **High** - direct attack vector; the cost is relatively low to the amount of funds that can be lost.
- **Medium** - only conditionally incentivized attack vector, but still relatively likely.
- **Low** - too many or too unlikely assumptions; provides little or no incentive.

Actions required by severity level

- **Critical** - client **must** fix the issue.
- **High** - client **must** fix the issue.
- **Medium** - client **should** fix the issue.
- **Low** - client **could** fix the issue.

Informational findings

Informational findings encompass recommendations to enhance code style, operations alignment with industry best practices, gas optimizations, adherence to documentation, standards, and overall contract design.

Informational findings typically have minimal impact on code functionality or security risk.

Evaluating all vulnerability types, including informational ones, is crucial for a comprehensive security audit to ensure robustness and reliability.

03 | Findings

High Severity Findings

[H-01] LPs For New Markets Are Vulnerable to Inflation Attack

> **Severity:** ***High risk*** *(Resolved)* *[PoC]*
>
> **Context:** ***liquidity-provider-v1.clar****:12-32,51-63* ***math-v1.clar****:53-64*

Description

The LP token uses a share-based accounting model where deposits mint `floor(assets * totalShares / totalAssets)` shares and redemptions return `floor(shares * totalAssets / totalShares)` assets.

While the `MINIMUM_INITIAL_DEPOSIT` check prevents the first deposit from being too small, it does not stop the LP share supply from later collapsing to a very small number through redemptions. An attacker can exploit this to steal from future depositors.

Setup

Bob collapses the share supply to 1:

1. Bob deposits 1000 assets, receives 1000 shares. Pool: `1000 assets / 1000 shares`.
2. Bob borrows and repays, causing at least 1 unit of interest. Pool: `1001 assets / 1000 shares`.
3. Bob redeems 999 shares, receives `floor(999 * 1001 / 1000) = 999` assets. Pool: `2 assets / 1 share`.

Net cost to Bob: 1 asset. He now holds 1 share worth 2 assets.

Inflation

Bob repeatedly inflates the value of his single share at zero net cost. At each step, with the pool at A `assets / 1 share`:

- Bob deposits `2A - 1` assets, minting `floor((2A-1) / A) = 1` share. Pool: `3A - 1 assets / 2 shares`.
- Bob redeems 1 share, receiving `floor((3A-1) / 2)` assets. Pool: `~3A/2 assets / 1 share`.
- Bob's cash outflow is ~A/2, but his remaining share's value increased by ~A/2. His total wealth (cash + share value) stays constant. The inflation costs Bob nothing; he is converting cash into share value temporarily.

The share price grows by ~1.5x per iteration: `2 -> 3 -> 4 -> 6 -> 9 -> 13 -> 19 -> 28 -> ...`, reaching any target in `O(log(target))` steps.

Theft

Bob inflates to A slightly above D/2 (where D is Alice's deposit). Alice mints 1 share. Pool after Alice's deposit: ~3D/2 assets / 2 shares. Bob redeems for 3D/4, profiting D/4. Alice can redeem her share for 3D/4, losing 25% of her deposit. Optionally, Bob can also deposit to get more shares before Alice's deposit in order to steal a higher percentage of her deposit.

In both cases, Bob's inflation capital is fully recovered. The only real cost is transaction fees for the ~30 iterations needed to reach a practical target.

This issue is not limited to protocol initialization. If all LP shares are ever redeemed and the pool returns to a 0 assets / 0 shares state, the next depositor effectively becomes the new initial depositor. Once a small amount of interest accrues again, the same share-supply collapse and inflation process can be repeated at any point in the protocol's lifetime. The attack also does not strictly require frontrunning: once the share price is inflated, the manipulated state persists and any future depositor is affected.

Recommendation

Introduce a permanent safeguard against extremely small LP share supplies. This can be done by adding an initialization requirement before allowing deposits to work. The initialization logic would require an initial dust deposit to a burn address before allowing normal deposits.

This ensures the total share supply can never fall below the minimum shares amount, preventing the pool from reaching a manipulable low-supply state.

There are implementation subtleties that need to be accounted for:

- initialization may be aware of old deployments, in the case of upgrades
- the burn address must be inaccessible
- a read count penalty on each deposit is an acceptable trade-off (for checking initialization status)
- upgrades and new deployments would require to first call the "initialization" logic beforehand

Example implementation:

```
 ;; ERRORS
 (define-constant ERR-INTEREST-PARAMS (err u10000))
-(define-constant ERR-ASSET-TOO-LOW (err u10001))
+(define-constant ERR-NOT-INITIALIZED (err u10001))
+(define-constant ERR-STATE-MALFORMED (err u10002))

 ;; CONSTANTS
 (define-constant SUCCESS (ok true))
 (define-constant MINIMUM_INITIAL_DEPOSIT u1000) ;; 0.001 USDC
    (6 decimals) - enforced only on first deposit
+(define-constant NULL_ADDRESS (unwrap-panic
   (principal-construct? (if is-in-mainnet 0x16 0x1a)
   0x0000000000000000000000000000000000000000)))
+
+;; LOCAL CACHE
+(define-data-var initialized bool false)
```

```
 ;; PUBLIC FUNCTIONS
+
+(define-public (initialize)
+  (let (
+      (total-lp-supply (unwrap! (contract-call? .state-v1
   get-total-supply) ERR-STATE-MALFORMED))
+    )
+    (var-set initialized true)
+    (try! (if (is-eq total-lp-supply u0)
+        (deposit MINIMUM_INITIAL_DEPOSIT NULL_ADDRESS)
+        SUCCESS
+    ))
+    (print {
+      action: "initialized",
+      caller: contract-caller
+    })
+    SUCCESS
+))
+
+
 (define-public (deposit (assets uint) (recipient principal))
   (begin
     (try! (accrue-interest))
     (let (
         (lp-params (contract-call? .state-v1 get-lp-params))
-        (total-assets (get total-assets lp-params))
         (shares (contract-call? .math-v1 convert-to-shares
           lp-params assets false))
       )
-      (try! (if (and (is-eq total-assets u0) (< assets
   MINIMUM_INITIAL_DEPOSIT)) ERR-ASSET-TOO-LOW SUCCESS))
+      (asserts! (var-get initialized) ERR-NOT-INITIALIZED)
       (try! (contract-call? .withdrawal-caps-v1 lp-deposit
         assets))
       (try! (contract-call? .state-v1 add-assets
         contract-caller recipient assets shares))
       (print {
```

Note: the example implementation above presumes already deployed vaults have already transferred shares to the burn address. Existing Granite markets are vey unlikely to every reach a point where all shares are redeemed, making this issue reproachable, but for extra paranoia, in case it was not already done, the Granite team can send dust LPs to the burn address to ensure no such case can ever happen on existing markets, even in extreme black swan type events that result in a full market LP bank-run.

Resolution

Resolved, the recommended fix was implemented in **PR#53**.

Medium Severity Findings

[M-01] Interest Dust Assigned to Staking Interest Causes Repay and Liquidation Underflows

> **Severity:** *Medium risk* *(Resolved)* *[PoC]*
>
> **Context:** ***borrower-v1.clar***:99-103 ***liquidator-v1.clar***:285-290,539-543

Description

When a borrower repays or is liquidated, the interest portion of the payment is split among three buckets: LP, protocol, and staking. The LP and protocol shares are computed with floored division, and the staking share receives the remainder:

```
(lp-part (contract-call? .math-v1 safe-div (* interest-part
    lp-open-interest-without-principal) open-interest-without-principal))
(protocol-part (contract-call? .math-v1 safe-div (* interest-part (get
    protocol-open-interest interest-params)) open-interest-without-principal))
(staked-part (contract-call? .math-v1 safe-sub interest-part (+ lp-part
    protocol-part)))
```

The `staked-part` is then subtracted from `staked-open-interest` in `state-v1` using raw subtraction:

```
(var-set open-interest {
  lp-open-interest: (- (get lp-open-interest open-interest-data) lp-part),
  staked-open-interest: (- (get staked-open-interest open-interest-data)
      staked-part),
  protocol-open-interest: (- (get protocol-open-interest open-interest-data)
      protocol-part)
})
```

When no LP tokens are staked, `staking-reward-percentage` is 0, so interest accrual never adds anything to `staked-open-interest`, keeping it at 0. However, the remainder logic can still assign a non-zero `staked-part`. This causes an arithmetic underflow when `state-v1` tries to subtract it.

Consider the following scenario:

1. No LP tokens are staked, so `staked-open-interest = 0`.
2. Interest has accrued to LP and protocol only, e.g. `lp-open-interest-without-principal = 7`, `protocol-open-interest = 2`, total = 9.
3. A borrower repays with `interest-part = 100`.
4. `lp-part = floor(100 * 7 / 9) = 77`, `protocol-part = floor(100 * 2 / 9) = 22`.
5. `staked-part = 100 - 77 - 22 = 1` (rounding dust).
6. `state-v1::update-repay-state` executes `(- 0 1)`, which underflows and reverts the entire transaction.

This is not limited to dust-sized repayments. Any repay or liquidation where the LP/protocol interest ratio produces a flooring remainder will revert. The same remainder-to-staked pattern and raw subtraction appear in all three state update paths: `update-repay-state` (line 751), `update-liquidate-collateral-state` (line 841), and `socialize-user-bad-debt` (line 921).

The practical impact is a DoS on repayments and liquidations whenever no LP tokens are staked. Blocked liquidations are particularly concerning since they allow bad debt to accumulate.

Recommendation

Revert the L-4/L-18 remainder-based assignment and return to computing `staked-part` proportionally with `math-v1::safe-div`, the same way `lp-part` and `protocol-part` are computed. This applies to all three locations: `borrower-v1` (line 103), `liquidator-v1` (lines 290 and 543).

Example implementation:

```
-(staked-part (contract-call? .math-v1 safe-sub interest-part (+
   lp-part protocol-part)))
+(staked-part (contract-call? .math-v1 safe-div (* interest-part
   (get staked-open-interest interest-params))
   open-interest-without-principal))
```

When `staked-open-interest` is 0, `safe-div` returns 0, eliminating the underflow entirely.

The trade-off is that up to 2 units of interest dust per repayment (0.000002 USDC) go unassigned to any bucket, which is the original behavior the L-4/L-18 fix tried to address. This dust leak is economically negligible and far preferable to a DOS on repayments and liquidations.

Resolution

Resolved, the recommended fix was implemented in **PR#54**.

[M-02] Updating Pyth Token Feeds Has no Timelock Enforcement

> **Severity:** *Medium risk (Resolved)*
>
> **Context:** *governance-v1.clar pyth-adapter-v1.clar*

Description

`ACTION_UPDATE_PYTH_TOKEN_FEED` (u26) is NOT included in the `time-locked` map at lines 1310-1335. A compromised governance multisig can instantly change any collateral token's Pyth oracle price feed ID to a feed returning a manipulated price. This enables mass liquidation or unlimited borrowing against worthless collateral.

The `initiate-proposal-to-update-pyth-feed` function at lines 1267-1286 creates a proposal and immediately tries to execute it:

```
(define-public (initiate-proposal-to-update-pyth-feed
    (token principal) (feed (buff 32)) (max-confidence-ratio uint) (expires-in
       uint))
```

```
    (let (
      (proposal-nonce (var-get next-proposal-nonce))
      (proposal-id (keccak256 (unwrap! (to-consensus-buff? {
        sender: contract-caller,
        nonce: proposal-nonce,
        action: ACTION_UPDATE_PYTH_TOKEN_FEED,
        expires-in: expires-in,
        data: {
          feed: feed,
          max-confidence-ratio: max-confidence-ratio
        }
      }) ERR-FAILED-TO-GENERATE-PROPOSAL-ID)))
    )
    (try! (create-proposal proposal-id ACTION_UPDATE_PYTH_TOKEN_FEED expires-in))
    (map-set update-pyth-feed proposal-id
      {token: token, feed: feed, max-confidence-ratio: max-confidence-ratio})
    (try! (execute-if-approve-threshold-met proposal-id))
    (ok proposal-id)
))
```

There are two issues:

Issue 1 - Missing timelock

Since u26 is absent from the `time-locked` map, `get-proposal-execution-time` at line 687 returns `stacks-block-height` (immediate execution). The proposal executes in the same block the threshold is met. Oracle feed changes are among the most dangerous governance actions - they directly determine all collateral valuations and liquidation thresholds.

Issue 2 - Token parameter excluded from proposal-id hash

At lines 1270-1279, the `proposal-id` hash only includes `feed` and `max-confidence-ratio` but NOT `token`. The `token` parameter is only stored in the `update-pyth-feed` map at line 1282. Other multisig members approving the proposal cannot cryptographically verify which token's feed will be changed - they must trust the proposer. A malicious proposer could announce "updating feed for token X" while actually targeting token Y, since both produce the same proposal-id hash if the feed and confidence ratio match.

When executed, it calls `execute-update-pyth-token-price-feed` at line 331, which forwards to `update-price-feed-id` in pyth-adapter-v1.clar at line 33:

```
(define-public (update-price-feed-id (token principal) (new-feed-id (buff 32))
    (max-confidence-ratio uint))
  (begin
    (asserts! (is-eq (contract-call? .state-v1 get-governance) contract-caller)
        ERR-NOT-AUTHORIZED)
    (map-set price-feeds token {
      feed-id: new-feed-id,
      max-confidence-ratio: max-confidence-ratio
    })
    SUCCESS
))
```

Attack path:

1. Attacker controls 60%+ of governance multisig (e.g., 2 of 3 signers)
2. Attacker creates a proposal to update sBTC's Pyth price feed ID to a feed for a near-worthless token
3. Second compromised signer approves. Threshold met, proposal executes INSTANTLY (no time-lock)
4. All sBTC-collateralized positions now read the worthless token's price as their collateral value
5. All sBTC positions become instantly liquidatable at near-zero collateral valuation
6. Attacker liquidates these positions, acquiring sBTC collateral at a massive discount
7. Alternatively: attacker points a cheap token's feed to a high-value feed, deposits cheap collateral, and borrows maximum USDC against the inflated valuation

Impact: Enables complete protocol theft via oracle manipulation. All positions using the targeted collateral token can be liquidated or the attacker can borrow unlimited USDC. The missing timelock removes the only detection window available to honest signers and the community.

Recommendation

1. Add `ACTION_UPDATE_PYTH_TOKEN_FEED` to the `time-locked` map:

```
 (map-set time-locked ACTION_ALLOW_ANY_CONTRACT_FLASH_LOAN true)
+(map-set time-locked ACTION_UPDATE_PYTH_TOKEN_FEED true)
```

2. Include the `token` parameter in the proposal-id hash so approving signers can cryptographically verify which token's feed is being changed:

```
 (proposal-id (keccak256 (unwrap! (to-consensus-buff? {
   sender: contract-caller,
   nonce: proposal-nonce,
   action: ACTION_UPDATE_PYTH_TOKEN_FEED,
   expires-in: expires-in,
   data: {
     feed: feed,
-    max-confidence-ratio: max-confidence-ratio
+    max-confidence-ratio: max-confidence-ratio,
+    token: token
   }
 }) ERR-FAILED-TO-GENERATE-PROPOSAL-ID)))
```

Resolution

Resolved, the recommended fix was implemented in **PR#55**.

[M-03] Staking Wipe-Out Detection Gap and Post-Wipe LP Token Leak

> **Severity:** *Medium risk* *(Resolved)* *[PoC]*
>
> **Context:** ***staking-v1.clar***:*202-239,189-199* ***borrower-v1.clar***:*128* ***liquidator-v1.clar***:*326,565*

Description

The staking module tracks LP tokens in two buckets: active (`total-lp-tokens-staked`) and withdrawal queue (`unfinalized-withdrawals.lp-tokens`). When bad debt is socialized, `staking-v1::slash-total-staked-lp-tokens` slashes LP tokens and sets a permanent `staking-wiped-out` flag if the slash consumes everything. Two related bugs allow value to leak into the staking module after it should be dead.

Bug A: Floor-division wipe-detection gap. The wipe-out check uses an equality test against the total:

```
(if (is-eq lp-tokens total-staked-lp-tokens)
  (begin
    (var-set staking-wiped-out true)
    ...
  )
  ...
)
```

Inside the same function, the slash is split between active and queue using floor division:

```
(withdrawal-lp-tokens-to-slash (/ (* lp-tokens withdrawal-lp-tokens)
    total-staked-lp-tokens))
(active-staked-lp-tokens-to-slash (- lp-tokens withdrawal-lp-tokens-to-slash))
```

When the withdrawal queue is small relative to total, `withdrawal-lp-tokens-to-slash` rounds to zero via floor division. The entire `lp-tokens` amount is then deducted from active. If `lp-tokens` is close to but less than `total-staked-lp-tokens`, active reaches zero while the queue retains dust. The `is-eq` check fails (the amounts are not equal), so `staking-wiped-out` never trips.

Example: active = 999, queue = 1, total = 1000. Slash of 999:

- `withdrawal-lp-tokens-to-slash = floor(999 * 1 / 1000) = 0`
- `active-staked-lp-tokens-to-slash = 999`
- Post-slash: active = 0, queue = 1. `(is-eq 999 1000)` = false. No wipe-out.

The staking module appears alive with zero active tokens and a dust-sized queue residue.

Bug B: Post-wipe LP token leak. Even when `staking-wiped-out` correctly fires, the callers (`borrower-v1::repay`, `liquidator-v1::execute-liquidation`, and `liquidator-v1::socialize-bad-debt`) do not check the flag. On every repay or liquidation:

1. Interest is split into LP, protocol, and staked portions. If `staked-open-interest > 0` (which persists from before the wipe), `staked-part > 0`.
2. `staked-lp-tokens = convert-to-shares(staked-part)` computes how many LP tokens to mint for the staked interest.
3. `state-v1::update-repay-state` mints these LP tokens into the staking contract via `ft-mint?`.
4. `staking-v1::increase-lp-staked-balance` increments `total-lp-tokens-staked`, making the dead module's active counter non-zero again.

This is self-reinforcing: the non-zero active counter causes `staking-reward-percentage > 0` on the next interest accrual, which routes more interest into `staked-open-interest`, which produces a larger `staked-part` on the next repay.

Zombie staked-LP-token holders (from before the wipe) can extract the phantom LP tokens via `initiate-unstake → finalize-unstake → liquidity-provider-v1::redeem`, converting them to real USDC.

Note: both issues were identified by the developers during the audit.

Recommendation

Two changes are needed:

1. **Fix wipe-out detection**: trigger `staking-wiped-out` when the post-slash active counter reaches zero, regardless of queue residue. Add an early-return guard for zero inputs (division-by-zero defense).
2. **Guard callers against post-wipe minting**: in `borrower-v1::repay`, `liquidator-v1::execute-liquidation`, and `liquidator-v1::socialize-bad-debt`, check `staking-v1::is-staking-wiped-out`. When wiped, zero out only `staked-lp-tokens` (to prevent LP token minting into the dead module) while keeping `staked-part` unchanged so that `state-v1` correctly reduces `staked-open-interest`. Also make `increase-lp-staked-balance` a silent no-op when wiped, as defense in depth.

Important: the `staked-part` passed to `state-v1` must NOT be zeroed, and `lp-part` must NOT absorb the staked interest. `state-v1::update-repay-state` uses raw subtraction (`-`, not `safe-sub`) to deduct each component from its corresponding `open-interest` bucket (line 749-753). Routing `staked-part` into `lp-part` causes `lp-open-interest` to underflow when the inflated LP deduction exceeds the LP bucket balance, reverting all repays and liquidations post-wipe. The correct approach is to keep the open-interest accounting unchanged (each bucket reduced by its proportional share) and only suppress the LP token operations (`ft-mint?` and `increase-lp-staked-balance`).

Example implementation for `staking-v1::slash-total-staked-lp-tokens`:

```
 (define-public (slash-total-staked-lp-tokens (lp-tokens uint))
-  (let (
-      (total-staked-lp-tokens (get-total-staked-lp-tokens))
+  (begin
+    (try! (contract-call? .state-v1 is-allowed-contract
   contract-caller))
+    (if (or (is-eq lp-tokens u0) (is-eq
   (get-total-staked-lp-tokens) u0))
+      SUCCESS
+      (let (
+          (total-staked-lp-tokens (get-total-staked-lp-tokens))
+          ...
+          (new-active (contract-call? .math-v1 safe-sub
   (var-get total-lp-tokens-staked)
   active-staked-lp-tokens-to-slash))
```

```
+         )
+         (var-set total-lp-tokens-staked new-active)
          ...
-     (if (is-eq lp-tokens total-staked-lp-tokens)
-       (begin
-         (var-set staking-wiped-out true)
+         (if (is-eq new-active u0)
+           (begin
+             (var-set staking-wiped-out true)
+             ...
```

Example implementation for callers (same pattern in all three):

```
+         (is-wiped (contract-call? .staking-v1
    is-staking-wiped-out))
+         (effective-staked-lp-tokens (if is-wiped u0
    staked-lp-tokens))
        )
        ...
        (try! (contract-call? .state-v1 update-repay-state {
          lp-part: (+ principal-part lp-part),
          protocol-part: protocol-part,
          staked-part: staked-part,
-         staked-lp-tokens: staked-lp-tokens,
+         staked-lp-tokens: effective-staked-lp-tokens,
          ...
        }))
-       (try! (contract-call? .staking-v1
    increase-lp-staked-balance staked-lp-tokens))
+       (try! (contract-call? .staking-v1
    increase-lp-staked-balance effective-staked-lp-tokens))
```

With this approach, `state-v1` correctly reduces each `open-interest` bucket by its proportional share (no underflow), while `staked-lp-tokens = 0` prevents `ft-mint?` from minting LP tokens into the dead module (the `(> staked-lp-tokens u0)` guard at state-v1.clar#L757 takes the SUCCESS branch). The `staked-open-interest` residual properly drains to zero as debts are repaid, cleaning up the accounting without leaking value.

Resolution

Resolved, the recommended fix was implemented in **PR#57**.

Low Severity Findings

[L-01] New Depositor Value Extraction After Catastrophic Socialization Zeroes Total Assets

> **Severity:** *Low risk (Resolved)*
>
> **Context:** *linear-kinked-ir-v1.clar:123-124* *linear-kinked-ir-utility.clar:69-71* *math-v1.clar:53-57*

Description

The utilization calculation was changed from guarding on `(+ total-assets open-interest) > 0` to `total-assets > 0`:

```
;; Old (division-by-zero bug when total-assets = 0 but open-interest > 0)
(if (> (+ total-assets open-interest) u0) (/ (* open-interest one-12) total-assets)
    u0)

;; New (correctly guards the denominator)
(if (> total-assets u0) (/ (* open-interest one-12) total-assets) u0)
```

This fix is necessary and correct: the old code would panic on division by zero when `total-assets = 0` and `open-interest > 0`, bricking the protocol after catastrophic socialization. The new code returns 0 utilization and allows the protocol to continue operating.

However, allowing the protocol to continue in the `total-assets = 0` state exposes a downstream issue in the LP share accounting. During `state-v1::socialize-user-bad-debt`, `state-v1::slash-staked-lp-tokens` only burns staked LP tokens and `reduce-total-assets-and-borrowable-balance` clamps `total-assets` to zero. LP tokens held by non-staked liquidity providers survive socialization, creating a state where `total-assets = 0` but `total-shares > 0` (the surviving LP token supply).

The share conversion function `convert-to-shares` has a 1:1 fallback when `total-assets = 0`:

```
(define-read-only (convert-to-shares (asset-params {total-assets: uint,
    total-shares: uint}) (assets uint) (round-up bool))
  (if (is-eq (get total-assets asset-params) u0)
    assets  ;; 1:1 fallback, ignores existing total-shares
    (divide round-up (* assets (get total-shares asset-params)) (get total-assets
        asset-params))
))
```

This fallback was designed for the first-ever deposit (both `total-assets` and `total-shares` are 0), but it also fires in the post-socialization state where `total-shares > 0`. A new deposit in this state causes immediate value loss:

1. Socialization zeros `total-assets`. Surviving LP supply `S = 10,000` (from non-staked LPs).
2. Alice deposits 1,000 USDC in the same block (interest accrual: `elapsed = 0`, no accrual, `total-assets` stays 0).

3. `convert-to-shares({total-assets: 0, total-shares: 10000}, 1000, false)` returns `1000` (1:1 fallback).
4. `add-assets` mints 1,000 LP tokens to Alice, sets `total-assets = 1000`.
5. State: `total-shares = 11,000`, `total-assets = 1,000` (entirely from Alice's deposit).
6. Alice's value: `1000 / 11000 * 1000 = ~91` USDC. She lost ~909 USDC.
7. The old shares capture `10000 / 11000 * 1000 = ~909` USDC of Alice's deposit. Old LP holders can withdraw this since `free-liquidity = 1000` (real USDC).

If `base-ir > 0` and the deposit occurs on a later block, interest accrual first pushes `total-assets` to a small nonzero value, and `convert-to-shares` uses the ratio formula instead. Alice would then receive a proportionally large share count, making the deposit fair. So the value extraction is limited to same-block deposits after socialization, or pools where `base-ir = 0`.

Additionally, when `total-assets = 0` but `open-interest > 0` and `base-ir > 0`, each interest accrual adds phantom `lp-interest` to `total-assets` (linear-kinked-ir-v1.clar:117). This creates a `total-assets` value not backed by actual USDC. The `free-liquidity` check in `remove-assets` (state-v1.clar:143) prevents extraction of phantom value, but LP share prices would display misleading nonzero values.

Recommendation

Block deposits when `total-assets = 0` but LP token supply is nonzero. This state indicates catastrophic socialization requiring governance intervention, not new deposits.

Example implementation:

```
 (define-public (deposit (assets uint) (recipient principal))
   (begin
     (try! (accrue-interest))
     (let (
         (lp-params (contract-call? .state-v1 get-lp-params))
         (total-assets (get total-assets lp-params))
         (shares (contract-call? .math-v1 convert-to-shares
            lp-params assets false))
       )
       (try! (if (and (is-eq total-assets u0) (< assets
          MINIMUM_INITIAL_DEPOSIT)) ERR-ASSET-TOO-LOW SUCCESS))
+      (asserts! (or (> total-assets u0) (is-eq (get
   total-shares lp-params) u0)) ERR-ASSET-TOO-LOW)
       (try! (contract-call? .withdrawal-caps-v1 lp-deposit
          assets))
       (try! (contract-call? .state-v1 add-assets
          contract-caller recipient assets shares))
```

The added assertion ensures deposits are only allowed when either the pool has assets (normal operation) or the LP supply is zero (first deposit / clean state). If `total-assets = 0` and `total-shares > 0`, the deposit reverts, preventing the share dilution.

Notes on recovery from the blocked state

If other borrowers remain in the pool (`open-interest > 0`), the protocol self-heals. Any repayment or liquidation routes the LP portion back into `total-assets`, pushing it above zero and unblocking deposits without governance action.

If the socialized borrower was the only borrower (`open-interest = 0`), no interest accrues and no repayments can restore `total-assets`; the pool is effectively dead. Recovery in this case requires governance intervention (contract migration or a dedicated function to inject assets into `total-assets`).

We do not recommend an intentional fix for this second case given how narrow the preconditions are (single-borrower catastrophic socialization), but the team should be aware and prepared to upgrade the `liquidity-provider-v1` contract if the situation arises.

Resolution

Resolved, the recommended fix was implemented in **PR#58**.

[L-02] Governance Threshold Calculated at Execution Time With Live Member Count Enables Retroactive Threshold Manipulation

> **Severity:** *Low risk* *(Resolved)*
>
> **Context:** *governance-v1.clar*

Description

The governance module uses a 60% approval threshold to execute proposals. The threshold check in `approve-threshold-met` calculates the percentage using the CURRENT member count from `meta-governance-v1`, not the count at proposal creation time:

```
(define-private (approve-threshold-met (proposal-id (buff 32)))
  (let (
      (proposal (unwrap! (map-get? governance-proposal proposal-id)
          ERR-UNKNOWN-PROPOSAL))
      (approve-count (get approve-count proposal))
      (total-count (contract-call? .meta-governance-v1 governance-multisig-count))
      (percentage (/ (* approve-count u100) total-count))
    )
    (ok (>= percentage THRESHOLD))
))
```

The `approve-count` is fixed at the time of voting, but `total-count` is read live from `meta-governance-v1::governance-multisig-count`. Changes to the multisig membership between proposal creation and execution alter whether the threshold is met, without any new votes being cast.

Additionally, line 1181 blocks new votes after `execute-at` is set:

```
(asserts! (is-none (get execute-at proposal)) ERR-CANNOT-VOTE)
```

This creates a window where member count changes during the timelock period can retroactively change the outcome with no recourse.

Scenario 1 - Lowering threshold (proposal passes without sufficient votes):

1. 5-member multisig. 60% threshold requires 3 approvals.
2. Proposal X receives 2 approvals (40% - insufficient).
3. Meta-governance removes 2 members through a separate proposal (now 3 members total).
4. Proposal X is re-evaluated: `2/3 = 66% >= 60%` - threshold now met.
5. Proposal X can be executed despite never receiving 60% approval from the original quorum.

Scenario 2 - Blocking execution (approved proposal fails):

1. 3-member multisig. 2 members approve a timelocked proposal (66% >= 60%).
2. `execute-at` is set. Timelock starts (24 hours).
3. During the 24-hour timelock, a meta-governance proposal adds 2 new members (now 5 total).
4. At execution time: `approve-threshold-met` recalculates: `2/5 = 40% < 60%`.
5. Execution silently fails (the `if threshold` branch in `execute-if-approve-threshold-met` takes the SUCCESS path without executing).
6. New members cannot vote because line 1181 blocks voting after `execute-at` is set.
7. Proposal is stuck: it cannot be executed (threshold not met) and cannot receive new votes (voting blocked).
8. Proposal remains stuck until expiration.

Recommendation

Snapshot the member count at proposal creation time and use the stored value for threshold calculation.

```
;; In create-proposal, store the current member count:
 (map-set governance-proposal proposal-id {
   action: action,
   approve-count: u1,
   deny-count: u0,
+  member-count: (contract-call? .meta-governance-v1
   governance-multisig-count),
   expires-at: (+ stacks-block-height expires-in),
   closed: false,
   executed: false,
   execute-at: none,
 })

;; In approve-threshold-met, use stored count:
 (define-private (approve-threshold-met (proposal-id (buff 32)))
   (let (
       (proposal (unwrap! (map-get? governance-proposal
          proposal-id) ERR-UNKNOWN-PROPOSAL))
       (approve-count (get approve-count proposal))
-      (total-count (contract-call? .meta-governance-v1
   governance-multisig-count))
+      (total-count (get member-count proposal))
```

```
        (percentage (/ (* approve-count u100) total-count))
      )
      (ok (>= percentage THRESHOLD))
 ))

;; Similarly update deny-threshold-met:
 (define-private (deny-threshold-met (proposal-id (buff 32)))
   (let (
       (proposal (unwrap! (map-get? governance-proposal
          proposal-id) ERR-UNKNOWN-PROPOSAL))
       (deny-count (get deny-count proposal))
-      (total-count (contract-call? .meta-governance-v1
   governance-multisig-count))
+      (total-count (get member-count proposal))
       (percentage (/ (* deny-count u100) total-count))
      )
      (ok (>= percentage THRESHOLD))
 ))
```

Resolution

Resolved, the recommended fix was implemented in **PR#59**.

[L-03] Disabling Interest Accrual Permanently Skips Pending Borrower Interest

> **Severity:** *Low risk (Resolved)*
>
> **Context:** *state-v1.clar:170-186 governance-v1.clar:1302-1308,357-366,340-355*

Description

`state-v1::set-interest-accrual-flag` advances `last-accrued-block-time` to the current block whenever `status` changes, without first calling `accrue-interest`:

```
(define-public (set-interest-accrual-flag (status bool))
  (begin
    (asserts! (is-governance) ERR-UNAUTHORIZED)
    (if (not (is-eq status (var-get interest-accrual-enabled)))
      (var-set last-accrued-block-time
        (+ (unwrap-panic (get-stacks-block-info? time (- stacks-block-height u1)))
          STACKS_BLOCK_TIME))
      true
    )
    (var-set interest-accrual-enabled status)
    ...
```

Interest accrued between the previous checkpoint and the disable transaction is permanently forgiven, on unpause the checkpoint resumes from the disable timestamp, skipping the gap. Three governance paths reach this function:

1. **Guardian emergency pause**, governance-v1::guardian-pause-market → state-v1::pause-market → set-interest-accrual-flag(false).
2. **Governance market pause**,

   execute-state-set-market-flag(ACTION_SET_MARKET_PAUSE_FLAG) → state-v1::pause-market → set-interest-accrual-flag(false).
3. **Direct disable**,

   execute-state-set-feature-flag(ACTION_SET_INTEREST_ACCRUAL_FLAG, flag=false) → set-interest-accrual-flag(false).

The loss is bounded by the interval between the last accrual checkpoint (any borrow, repay, deposit, withdraw, liquidation, stake, or unstake triggers accrual) and the pause transaction, typically minutes to hours of LP, staker, and protocol-reserve interest. The path is governance-gated with no permissionless trigger.

Note: this issue was identified via Immunefi bug bounty.

Recommendation

state-v1 is immutable, so set-interest-accrual-flag and pause-market cannot be patched. However, governance-v1 is upgradable and already has a private accrue-interest function used by execute-update-interest-rate-params, execute-update-reward-rate-params, and execute-protocol-reserve-percentage for exactly this purpose, as such, fixes will be implemented in this contract.

Each of the mentioned execution path has a different operational constraint, so they should be handled separately:

Path	Suggested Fix	Rationale
1. Guardian pause	Best-effort accrue-interest (swallow errors)	Emergency pause must not be blockable, the state being escaped may itself cause accrual to fail
2. Governance pause	(try! (accrue-interest)) (hard-fail on error)	Planned downtime; governance can address the error and resubmit
3. Direct disable	(try! (accrue-interest)) (hard-fail on error)	Same as Path 2; governance can address and resubmit

Example implementation for Path 1 (guardian-pause-market):

```
(define-public (guardian-pause-market)
  (begin
    (try! (is-guardian contract-caller))
```

```
+    (match (accrue-interest) ok-result true err-result true)
     (try! (contract-call? .state-v1 pause-market))
     (try! (contract-call? .state-v1 set-staking-flag false))
     SUCCESS
 ))
```

Example implementation for Path 2 (`execute-state-set-market-flag`):

```
 (define-private (execute-state-set-market-flag (proposal-id
    (buff 32)) (action uint))
   (let ((cooldown (unwrap-panic (map-get? unpause-market-data
      proposal-id))))
     (asserts! (not (is-eq action ACTION_SET_MARKET_PAUSE_FLAG))
        (begin
+      (try! (accrue-interest))
       (try! (contract-call? .state-v1 pause-market))
       (contract-call? .state-v1 set-staking-flag false)))
     ...
```

Example implementation for Path 3 (`execute-state-set-feature-flag`):

```
 (define-private (execute-state-set-feature-flag (proposal-id
    (buff 32)) (action uint))
   (let (
       (data (unwrap-panic (map-get?
          set-market-feature-proposal-data proposal-id)))
       (flag (get flag data))
       (cooldown (get cooldown data))
     )
     ...
-    (asserts! (not (is-eq action
    ACTION_SET_INTEREST_ACCRUAL_FLAG)) (contract-call? .state-v1
    set-interest-accrual-flag flag))
+    (asserts! (not (is-eq action
    ACTION_SET_INTEREST_ACCRUAL_FLAG)) (begin
+      (try! (accrue-interest))
+      (contract-call? .state-v1 set-interest-accrual-flag
    flag)))
     ERR-INVALID-ACTION
 ))
```

Resolution

Resolved, the recommended fix was implemented in **PR#63**.

Informational Findings

[I-01] Redundant Reentrancy Guard in Flash Loan Due to Clarity VM Built-in Protection

> **Severity:** *Informational* *(Resolved)*
>
> **Context:** *flash-loan-v1.clar:26,91-93*

Description

The `flash-loan-v1::flash-loan` function uses a `flash-loan-in-progress` flag as a reentrancy guard:

```
(define-data-var flash-loan-in-progress bool false)

(define-public (flash-loan (amount uint) (callback <callback-trait>) (data
    (optional (buff 20480))))
  (let (...)
    (asserts! (not (var-get flash-loan-in-progress)) ERR-FLASH-LOAN-IN-PROGRESS)
    (var-set flash-loan-in-progress true)
    ;; ... callback execution ...
    (var-set flash-loan-in-progress false)
    SUCCESS
  )
)
```

This guard is redundant as the Clarity VM prevents the same function from appearing twice on the call stack, enforced in the Stacks VM itself.

The no execution path starting with the callback can re-enter `flash-loan` regardless of the flag, because the VM will abort the transaction if `flash-loan` is called again while already executing.

Note that Clarity does allow same-contract reentrancy for different functions, so the callback can call other functions on `flash-loan-v1` (e.g., read-only getters). However, calling `flash-loan` itself is blocked by the VM.

As a result, the `ERR-FLASH-LOAN-IN-PROGRESS` assertion can never trigger and the extra `var-set` and `var-get` calls wastes read and write budget.

Recommendation

There are two options that can be followed:

1. **Remove the flag entirely**

The VM guarantee makes it dead code. This saves gas on every flash loan call (two `var-set` operations and one `var-get` check).

1. **Expose it via a read-only function for external integrators**

The flag observable as `true` during the callback however no getter exists to retrieve it. Adding one would any third party contracts that are called from within the callback execution path to detect they are executing within a flash loan context.

Note, in this case, also remove the `ERR-FLASH-LOAN-IN-PROGRESS` error and assertion, since only setting the variable is revelant.

Example implementation:

```
(define-read-only (is-flash-loan-in-progress) (var-get flash-loan-in-progress))
```

We recommend to remove the flag (1), since it serves no purpose under Clarity's current execution model. If kept, at minimum expose it via a read-only function so it can have a potential utility (2).

Resolution

Resolved, the recommended fix was implemented in **PR#60**.

[I-02] Redundant safe-sub in calculate-repayment-info Denominator Computation

> **Severity:** *Informational* *(Resolved)*
>
> **Context:** *liquidator-v1.clar:378-381*

Description

The `liquidator-v1::calculate-repayment-info` function computes a `denominator` used to determine the liquidator's maximum repayment:

```
(denominator (contract-call? .math-v1 safe-sub
    SCALING-FACTOR
    (try! (safe-div (* (+ SCALING-FACTOR liquidation-discount)
        collateral-liquid-ltv) SCALING-FACTOR))
))
```

`safe-sub` clamps to 0 if the subtrahend exceeds `SCALING-FACTOR`. However, this condition is unreachable because `state-v1::update-collateral-settings` enforces an invariant via `is-valid-liqLTV-and-liqPremium` that guarantees the subtrahend is always strictly less than `SCALING-FACTOR`.

The validation (state-v1 line 424-428):

```
(define-private (is-valid-liqLTV-and-liqPremium (liquidation-ltv uint)
    (liquidation-premium uint))
  (let ((inverted-discount (/ (* scaling-factor scaling-factor) (+ scaling-factor
     liquidation-premium))))
    (asserts! (< liquidation-ltv inverted-discount) ERR-INVALID-PARAMS)
    SUCCESS
  )
)
```

This enforces `liquidation-ltv < floor(SCALING-FACTOR^2 / (SCALING-FACTOR + liquidation-premium))`, which is equivalent to:

```
(SCALING-FACTOR + liquidation-premium) * liquidation-ltv < SCALING-FACTOR^2
```

Since the inner expression in `calculate-repayment-info` computes `floor((SCALING-FACTOR + liquidation-discount) * collateral-liquid-ltv / SCALING-FACTOR)`, and the validation guarantees `(SCALING-FACTOR + liquidation-discount) * collateral-liquid-ltv < SCALING-FACTOR^2`, floor-dividing both sides by `SCALING-FACTOR` gives:

```
floor((SCALING-FACTOR + liquidation-discount) * collateral-liquid-ltv /
    SCALING-FACTOR) < SCALING-FACTOR
```

The subtracted is always strictly less than `SCALING-FACTOR`, so the subtraction can never underflow. Using `safe-sub` here silently clamps to 0 instead of panic reverting but an error revert will issued on `total-repay-amount` calculation, since the `safe-div` will revert with `ERR-DIVIDE-BY-ZERO`.

```
(total-repay-amount (try! (safe-div (* (- debt total-collaterals-liquid-value)
    SCALING-FACTOR) denominator)))
```

Recommendation

Replace `safe-sub` with a normal subtraction, since the state contract enforced invariant guarantees it cannot underflow.

Example fix:

```
- (denominator (contract-call? .math-v1 safe-sub
-     SCALING-FACTOR
-     (try! (safe-div (* (+ SCALING-FACTOR liquidation-discount)
   collateral-liquid-ltv) SCALING-FACTOR))
- ))
+ (denominator (-
+     SCALING-FACTOR
+     (try! (safe-div (* (+ SCALING-FACTOR liquidation-discount)
   collateral-liquid-ltv) SCALING-FACTOR))
+ ))
```

Resolution

Resolved, the recommended fix was implemented in **PR#60**.

[I-03] Full Debt Repayment Leaves Phantom Borrowed Amount Residual

> **Severity:** ***Informational*** *(Resolved)* *[PoC]*
>
> **Context:** ***borrower-v1.clar***:*90-108* ***liquidator-v1.clar***:*278-294,533-545*

Description

Debt accounting uses a share-based system where a user's current debt is derived from their shares:

```
current-debt = ceil(open-interest * user-debt-shares / total-debt-shares)
```

While `open-interest` grows monotonically via interest accrual and `total-debt-shares` is unchanged during accrual, the `open-interest / total-debt-shares` ratio can decrease by a fraction of a unit each time bad debt is socialized. This is because `state-v1::socialize-user-bad-debt` removes `ceil(OI * user-shares / total-shares)` from `open-interest` (ceiling rounds up by at most 1) while removing the exact `user-shares` from `total-debt-shares`, effectively eroding the ratio by up to `1 / (total-shares - user-shares)` per socialization event.

When socializations occur in separate blocks, this erosion is compensated by interest accrual (if interest accrual is enabled): `linear-kinked-ir-v1::accrue-interest` uses `math-v1::divide-round-up`, guaranteeing at least 1 unit of interest per block. Since each socialization's rounding error is strictly less than 1 unit, the net drift per block is always positive.

However, this compensation does not apply within a single block. When multiple liquidations are included in one block, `accrue-interest` returns early with no changes after the first transaction (`elapsed-block-time = 0`). The ceiling rounding from subsequent same-block socializations accumulates without interest compensation.

Note that `borrower-v1::borrow` allocates debt shares with `convert-to-debt-shares(debt-params, amount, true)` (ceiling), which gives the borrower slightly more shares than proportional. This creates a protective margin: their `current-debt = ceil(OI * shares / TDS)` is typically at least 1 unit above `borrowed-amount` once any interest has accrued. A single socialization erodes the OI/TDS ratio by at most `1 / remaining_TDS`, which translates to less than 1 unit of current-debt drop for any borrower holding less than half the total shares. Therefore, a single socialization cannot trigger the condition on its own. However, 2-3 same-block socializations can accumulate enough rounding erosion to overcome this margin, particularly for borrowers whose `OI * shares / TDS` is already close to an integer boundary.

A recently introduced defensive guard handles this edge case:

```
(interest-portion (if (>= current-debt borrowed-amount)
  (contract-call? .math-v1 calculate-interest-portions current-debt borrowed-amount
     repay-amount)
  {principal-part: repay-amount, interest-part: u0}
))
```

The fallback correctly assigns zero interest (there is none to distribute). However, the downstream accounting creates a ghost residual on full repay:

1. `borrower-v1::max-repay-amount` caps `repay-amount` at `current-debt` (e.g., 999)
2. Fallback sets `principal-part = repay-amount = 999`
3. `updated-borrowed-amount = math-v1::safe-sub(borrowed-amount, principal-part) = safe-sub(1000, 999) = 1`
4. `shares` consumed = all user shares → `debt-shares` becomes 0

Final position state: `{debt-shares: 0, borrowed-amount: 1}`. The user has no debt but a phantom principal of 1 remains in storage. This 1-unit residual also persists in `total-borrowed-`

`amount` globally, making `open-interest-without-principal` (used for interest distribution to LP/protocol/stakers) slightly smaller than it should be.

The triggering condition requires a specific but plausible combination: multiple bad-debt socializations within the same block (accumulating ceiling-rounding erosion without intermediate interest compensation) against a borrower whose `OI * shares / TDS` sits near an integer boundary. The practical impact is negligible: the resulting `{debt-shares: 0, borrowed-amount: 1}` position is functionally dead, and the 1-unit drift in `total-borrowed-amount` would need to accumulate past `open-interest` before affecting interest distribution, a gap that interest continuously widens.

Recommendation

The fix has two parts: cleaning up per-user `borrowed-amount` and cleaning up the global `total-borrowed-amount`. Both apply to `borrower-v1::repay`, `liquidator-v1::execute-liquidation`, and `liquidator-v1::socialize-bad-debt`.

Part 1 - Per-user `borrowed-amount` (`borrower-v1` and `liquidator-v1::execute-liquidation`)

Use `min(current-debt, borrowed-amount)` as the effective principal for both the interest-portion calculation and the `updated-borrowed-amount` derivation, so that a full repay zeroes out the user's position cleanly.

`borrower-v1.clar`:

```
- (interest-portion (if (>= current-debt borrowed-amount)
-  (contract-call? .math-v1 calculate-interest-portions
   current-debt borrowed-amount repay-amount)
-  {principal-part: repay-amount, interest-part: u0}
- ))
+ (effective-borrowed-amount (if (< current-debt
   borrowed-amount) current-debt borrowed-amount))
+ (interest-portion (contract-call? .math-v1
   calculate-interest-portions current-debt
   effective-borrowed-amount repay-amount))
  ...
- (updated-borrowed-amount (contract-call? .math-v1 safe-sub
   borrowed-amount principal-part))
+ (updated-borrowed-amount (contract-call? .math-v1 safe-sub
   effective-borrowed-amount principal-part))
```

When `current-debt < borrowed-amount`, `effective-borrowed-amount = current-debt`. On full repay `principal-part = repay-amount = current-debt`, so `updated-borrowed-amount = safe-sub(current-debt, current-debt) = 0`. Without the second change, the code still uses the original `borrowed-amount` (which exceeds `current-debt`) and produces a non-zero residual.

The same change applies identically to `liquidator-v1::execute-liquidation`.

This part does not apply to `liquidator-v1::socialize-bad-debt` because the user's position is already fully zeroed by `state-v1::socialize-user-bad-debt`, so the per-user `borrowed-amount` is cleaned up regardless.

Part 2 - Global `total-borrowed-amount` (all three locations)

`total-borrowed-amount` tracks the sum of all users' stored `borrowed-amount` (principal, not debt). When the edge case triggers, `principal-part` (= `current-debt`) is smaller than the user's stored `borrowed-amount`, so `updated-total-borrowed-amount = safe-sub(total-borrowed-amount, principal-part)` under-subtracts, leaving a 1-unit phantom in the global counter.

The fix differs by location because `borrower-v1::repay` and `liquidator-v1::execute-liquidation` can be partial (not all debt-shares consumed), while

`liquidator-v1::socialize-bad-debt` always fully removes the user.

`borrower-v1.clar` and `liquidator-v1::execute-liquidation`, conditional on full repay (debt-shares reaches 0):

```
- (updated-total-borrowed-amount (contract-call? .math-v1
   safe-sub total-borrowed-amount principal-part))
+ (updated-total-borrowed-amount (contract-call? .math-v1
   safe-sub total-borrowed-amount
+    (if (is-eq total-user-debt-shares u0) borrowed-amount
   principal-part)))
```

`liquidator-v1::socialize-bad-debt`, unconditional since the user is always fully removed:

```
- (updated-total-borrowed-amount (contract-call? .math-v1
   safe-sub total-borrowed-amount principal-part))
+ (updated-total-borrowed-amount (contract-call? .math-v1
   safe-sub total-borrowed-amount user-borrowed-amount))
```

For the `socialize-bad-debt` function, in the normal case (`current-debt >= borrowed-amount`), these changes are no-ops on full repay. When `repay-amount = current-debt`, the call to `math-v1::calculate-interest-portions`

```
(contract-call? .math-v1 calculate-interest-portions remaining-debt
   user-borrowed-amount remaining-debt)
```

splits the repayment as: `interest-accrued = current-debt - borrowed-amount`, `interest-part = divide-round-up(interest-accrued * current-debt / current-debt) = interest-accrued` (exact, no rounding), `principal-part = current-debt - interest-accrued = borrowed-amount`. So `principal-part` already equals the user's stored `borrowed-amount`. As such, in `socialize-bad-debt` this is guaranteed because the third argument to `calculate-interest-portions` is always `remaining-debt` (the full user debt), making it always a full repay: `principal-part = user-borrowed-amount`.

Resolution

Resolved, the recommended fix was implemented in **PR#60**.

[I-04] LP Incentives Authorization Check Renders On-Behalf-Of Parameter Ineffective

> **Severity:** *Informational* *(Resolved)*
>
> **Context:** *lp-incentives-v2.clar:122-125*

Description

The `lp-incentives-v2::claim-rewards` function accepts an optional `on-behalf-of` parameter, which previously enabled third parties to trigger reward claims for another user. The claimed STX was always transferred to the reward owner, not the caller, so third-party claiming did not allow reward theft.

A recently applied fix introduces an authorization check:

```
(user (default-to contract-caller on-behalf-of))
(auth-check (asserts! (or (is-eq contract-caller user) (is-none on-behalf-of))
    ERR-NOT-AUTHORIZED))
```

This assertion only passes when:

- `on-behalf-of` is none (user defaults to `contract-caller`), or
- `on-behalf-of` equals `contract-caller` (redundant, same result)

Any other value of `on-behalf-of` makes the first condition false (`contract-caller != other`) and the second false (`on-behalf-of` is not none), so the transaction reverts. The effective claimant is always `contract-caller`, making `on-behalf-of` dead code.

The previous capability for keepers, helper contracts, or other third parties to claim rewards on behalf of users has been removed. If permissionless third-party claiming was intentional, this change unnecessarily restricts existing functionality.

Recommendation

Clarify whether third-party claiming is intended behavior.

If rewards are intended to be self-claimed only, remove the `on-behalf-of` parameter entirely and consistently use `contract-caller` as the claimant.

If third-party claiming is intended, remove the authorization check, since rewards are transferred to the reward owner regardless of who initiates the claim.

Resolution

Resolved, the recommended fix was implemented in **PR#61**.

[I-05] Missing Elapsed Time Clamp in Refill Bucket Logic Allows Revert on Timestamp Regression

> **Severity:** *Informational* *(Resolved)*
>
> **Context:** *withdrawal-caps-v1.clar:77,88*

Description

The last protocol changes introduced elapsed time clamping across the codebase to guard against block timestamp regression. The pattern `(if (> time-now last-time) (- time-now last-time) u0)` prevents a uint underflow revert if `time-now` is ever less than the stored timestamp. This guard was applied consistently in three locations:

- `linear-kinked-ir-v1.clar:87` (interest accrual)
- `linear-kinked-ir-utility.clar:32` (utility interest accrual)
- `withdrawal-caps-v1.clar:88` (`decay-bucket-amount`)

However, `refill-bucket-amount` at line 77 was not updated and still uses a raw subtraction:

```
;; refill-bucket-amount (line 77) - NO guard
(elapsed (if (is-eq last-updated-at u0) refill-window (- time-now last-updated-at)))

;; decay-bucket-amount (line 88) - HAS guard
(elapsed (if (is-eq last-updated-at u0) decay-window (if (> time-now
    last-updated-at) (- time-now last-updated-at) u0)))
```

Both functions are called from the same `sync-lp-bucket` (line 95), `sync-debt-bucket` (line 121), and `sync-collateral-bucket` (line 146) dispatch logic, where `refill-bucket-amount` is used when `current-bucket < max-bucket` and `decay-bucket-amount` when `current-bucket >= max-bucket`.

Stacks block timestamps are derived from Bitcoin block timestamps, which follow the median-time-past rule: a block's timestamp must exceed the median of the prior 11 blocks but CAN be lower than the immediately preceding block's timestamp. This means small timestamp regressions between consecutive blocks are protocol-valid, though extremely rare in practice.

If a timestamp regression occurred, any bucket sync where the bucket is below its max (the common steady-state) would call `refill-bucket-amount`, hit the raw subtraction with `time-now < last-updated-at`, and revert due to uint underflow. This would temporarily block LP deposits, LP withdrawals, borrows, repayments, liquidations, and collateral operations that route through the withdrawal caps module, until the next block restores monotonic timestamps.

In contrast, if the bucket were above its max, `decay-bucket-amount` would handle the same regression gracefully by clamping elapsed to 0.

Recommendation

Apply the same clamping guard to `refill-bucket-amount` for consistency with the other three locations.

Example implementation:

```
 (define-private (refill-bucket-amount (last-updated-at uint)
    (time-now uint) (max-bucket uint) (current-bucket uint)
    (inflow uint))
   (let (
       (refill-window (get-refill-time-window))
-      (elapsed (if (is-eq last-updated-at u0) refill-window (-
   time-now last-updated-at)))
+      (elapsed (if (is-eq last-updated-at u0) refill-window (if
   (> time-now last-updated-at) (- time-now last-updated-at)
   u0)))
       (refill-amount (if (>= elapsed refill-window) max-bucket
          (/ (* max-bucket elapsed) refill-window)))
       (new-bucket (min (+ current-bucket refill-amount)
          max-bucket))
   )
     (+ new-bucket inflow)
 ))
```

Resolution

Resolved, the recommended fix was implemented in **PR#61**.

[I-06] Oracle Staleness Minimum of 60 Seconds Is Excessive for Stacks Block Times

> **Severity:** *Informational* *(Resolved)*
>
> **Context:** *pyth-adapter-v1.clar:15,54-55*

Description

The M-1 audit fix added governance bounds on the `time-delta` parameter, which controls how old a Pyth oracle price can be before it is rejected as stale:

```
(define-constant MINIMUM_TIME_DELTA u60) ;; 60 seconds

(define-public (update-time-delta (delta uint))
  (begin
    (asserts! (is-eq (contract-call? .state-v1 get-governance) contract-caller)
       ERR-NOT-AUTHORIZED)
    (asserts! (>= delta MINIMUM_TIME_DELTA) ERR-INVALID-TIME-DELTA)
    (asserts! (<= delta MAXIMUM_TIME_DELTA) ERR-INVALID-TIME-DELTA)
    ...
```

The minimum of 60 seconds was introduced to prevent governance from setting `time-delta` to 0, which would reject all non-future prices and freeze price-dependent operations. This is a valid concern, but the chosen floor of 60 seconds is excessive for a chain with ~5-second block times (although, in practice it oscillates between ~4.4-7.6 seconds, with a 30-day average of ~6.8 seconds). It means

governance cannot require oracle prices fresher than ~9-14 blocks, even during periods of high volatility where tighter freshness requirements would be desirable.

During a flash crash or rapid price movement, the difference between a 10-second-stale and a 60-second-stale price can be several percent. A borrower observing a collateral price drop on external markets has up to 60 seconds (at the tightest governance setting) where the protocol still accepts the pre-drop price as valid, potentially allowing them to borrow against or withdraw collateral at an inflated valuation before the Pyth feed catches up.

The `STACKS_BLOCK_TIME` constant (5 seconds) is already available in the contract. A minimum of 2-3 block times (10-15 seconds) would be sufficient to prevent the `time-delta = 0` freeze while preserving governance's ability to enforce tighter freshness during volatile conditions.

Recommendation

Lower `MINIMUM_TIME_DELTA` to allow governance to enforce tighter freshness. A value of 15 seconds (~2 blocks at average observed block times) is sufficient to prevent the `time-delta = 0` freeze while accommodating at least one Pyth update cycle.

Example implementation:

```
-(define-constant MINIMUM_TIME_DELTA u60)
+(define-constant MINIMUM_TIME_DELTA u15)
```

Note: deriving from `STACKS_BLOCK_TIME` (e.g., `(* u3 STACKS_BLOCK_TIME)`) is also an option, but that constant is fixed at 5 seconds and does not reflect actual block time variance (~4.4-7.6 seconds observed). A hardcoded value avoids a false sense of dynamic adaptation.

Resolution

Resolved, the recommended fix was implemented in **PR#61**.

[I-07] Unbounded Interest Rate Output Combined With Taylor Guard Can Theoretically Block Protocol

> **Severity:** *Informational* *(Resolved)*
>
> **Context:** *linear-kinked-ir-v1.clar:123-124,136-138,161-171*

Description

The audit fix added a Taylor input guard (L-7 fix) that reverts interest accrual when `rt` exceeds `MAX-TAYLOR-INPUT` (2.0 in 12-fixed):

```
(define-constant MAX-TAYLOR-INPUT u2000000000000) ;; 2.0

(define-read-only (compounded-interest (current-interest-rate uint)
    (elapsed-block-time uint))
  (let ((rt (get-rt-by-block current-interest-rate elapsed-block-time)))
    (asserts! (<= rt MAX-TAYLOR-INPUT) ERR-TAYLOR-INPUT-TOO-LARGE)
    (ok (taylor-6 rt))
))
```

Where `rt = rate * elapsed / seconds-in-year`. The guard is correct and necessary; without it, the Taylor-6 approximation produces silently wrong results for large inputs. However, `rate` is the output of `ir-calc`, which is unbounded when utilization exceeds 100%.

The utilization calculation is `open-interest * 10^12 / total-assets`. When `open-interest > total-assets` (stress conditions, post-socialization recovery), utilization exceeds `one-12`. The function `interest-util-geq-kink` scales linearly with utilization:

```
(define-private (interest-util-geq-kink (util-with-res uint))
  (+
    (/
      (+
        (* (var-get ir-slope-2) (- util-with-res (var-get utilization-kink)))
        (* (var-get ir-slope-1) (var-get utilization-kink))
      )
      one-12
    )
    (var-get base-ir)
))
```

There is no cap on `util-with-res`, so the output interest rate grows unboundedly. Combined with the Taylor guard, this creates a threshold beyond which `accrue-interest` reverts. Since every protocol operation (deposit, withdraw, borrow, repay, liquidate) calls `accrue-interest` first, exceeding this threshold bricks the protocol until the rate * elapsed product decreases.

The Taylor guard triggers when `rate * elapsed > MAX-TAYLOR-INPUT * seconds-in-year = 6.31 * 10^19`. Example thresholds for the maximum interaction gap before bricking (assuming `slope-2 = 5 * 10^13`, `kink = 0.8 * one-12`):

Utilization	Effective annual rate	Max interaction gap
150%	~4.5 * 10^13	~16 days
200%	~7.5 * 10^13	~9.7 days
300%	~1.35 * 10^14	~5.4 days

With `MAX-IR-PARAM = 10^14` (10,000% annual, the governance bound) at `slope-2`, these windows shrink by roughly half.

Mitigating factors:

1. **Market pause/unpause resets the timestamp.** `set-interest-accrual-flag` updates `last-accrued-block-time` to the current time whenever the flag is toggled. Both `pause-market` and `unpause-market` use this function, so a pause/unpause cycle cannot accumulate dangerous elapsed time.
2. **High-utilization markets have active participants.** Borrowers facing extreme rates and liquidators monitoring positions would interact regularly, keeping the elapsed time small.
3. **Governance can recover via timestamp reset.** Governance can call `set-interest-accrual-flag(false)` then `set-interest-accrual-flag(true)` to reset `last-accrued-block-time` to the current time (two separate transactions). This bypasses the

bricked `accrue-interest` because `execute-state-set-feature-flag` does not call `accrue-interest`. Note that lowering IR params via `update-ir-params` is NOT a viable recovery path: the governance execution at `governance-v1:428` calls `accrue-interest` first, which would also revert. The timestamp reset skips the interest that would have accrued during the bricked period.

4. **`MAX-IR-PARAM = 10^14` is very generous.** Realistic lending protocols use slope-2 values of 10^12 to 10^13 (100-1,000% annual above kink). At slope-2 = 10^12 and 200% utilization, the max interaction gap before bricking is ~970 days. The finding is only practical at aggressive param settings.

Recommendation

Cap utilization in `ir-calc` at a reasonable maximum (e.g., 10x or 1000%). This bounds the effective interest rate regardless of how distressed the pool becomes, preventing the Taylor guard from being reachable with reasonable elapsed times.

Example implementation:

```
+(define-constant MAX-UTILIZATION u10000000000000) ;; 10x
   (1000%) in 12-fixed
+
+(define-private (min (a uint) (b uint))
+  (if (<= a b) a b)
+)

 (define-read-only (utilization-calc (total-assets uint)
    (open-interest uint))
-  (if (> total-assets u0) (/ (* open-interest one-12)
   total-assets) u0)
+  (if (> total-assets u0) (min (/ (* open-interest one-12)
   total-assets) MAX-UTILIZATION) u0)
 )
```

Resolution

Resolved, the recommended fix was implemented in **PR#61**.

[I-08] Redundant safe-sub in slash-total-staked-lp-tokens

> **Severity:** *Informational* *(Resolved)*
>
> **Context:** *staking-v1.clar:211*

Description

`staking-v1::slash-total-staked-lp-tokens` uses `math-v1::safe-sub` (clamp to 0) when reducing the active staked LP token balance:

```
(var-set total-lp-tokens-staked
  (contract-call? .math-v1 safe-sub (var-get total-lp-tokens-staked)
     active-staked-lp-tokens-to-slash))
```

The subtraction cannot underflow through the existing call path. The only caller is `liquidator-v1::socialize-bad-debt` (line 559), which passes `burned-staking-lp-tokens` originating from `state-v1::slash-staked-lp-tokens` (line 872). That function caps the value at `get-total-staked-lp-tokens` (active + withdrawal). Within `slash-total-staked-lp-tokens`:

```
active-staked-lp-tokens-to-slash = lp-tokens - (lp-tokens * withdrawal / total)
                                 = lp-tokens * active / total
```

Since `lp-tokens <= total` (from the caller cap), this gives `active-staked <= active`, so the subtraction `active - active-staked` never underflows.

The `safe-sub` is therefore redundant and introduces execution costs overhead.

Recommendation

Revert to the raw subtraction to remove the unnecessary cross-contract call overhead.

Example implementation:

```
-(var-set total-lp-tokens-staked (contract-call? .math-v1
   safe-sub (var-get total-lp-tokens-staked)
   active-staked-lp-tokens-to-slash))
+(var-set total-lp-tokens-staked (- (var-get
   total-lp-tokens-staked) active-staked-lp-tokens-to-slash))
```

Resolution

Resolved, the recommended fix was implemented in **PR#62**.

[I-09] Miscellaneous Code Quality Issues

> **Severity:** *Informational* *(Resolved)*
>
> **Context:** *flash-loan-v1.clar:13* *liquidator-v1.clar:21,301-302,503-508* *withdrawal-caps-v1.clar:271*

Description

Several minor code quality issues were identified across the changed contracts. None affect protocol correctness or security individually, but they reduce code clarity, hinder debugging, or leave dead code that complicates future maintenance.

1. **Inconsistent error constant naming** (`flash-loan-v1:13`):

   `ERR_CONTRACT_NOT_ALLOWED` uses underscores. Every other error constant across the codebase uses hyphens (`ERR-NOT-AUTHORIZED`, `ERR-INVALID-FEE`, etc.). The constant is referenced at line 92.

2. **Unused error constant** (`liquidator-v1:21`): `ERR-LIQUIDATED-TOO-MUCH` (`u30004`) is defined but never referenced anywhere in the contract or codebase.
3. **Inaccurate comment and audit issue reference** (`liquidator-v1:301`): The comment `L-36: require at least 6 blocks (~1 min) between borrowing and liquidation` has two problems. It references a prior audit issue ID that should not persist in production code. The time estimate is also inaccurate: `STACKS_BLOCK_TIME` is hardcoded to 5 seconds, but real block times from the Hiro API shows an average of 6.83s (30d). At 6.83s/block, 6 blocks is ~41 seconds, not ~1 minute. The same issue ID appears at line 256.
4. **Magic number for block delay** (`liquidator-v1:302`): The `u6` block delay in the liquidation timing check is a magic number. It should be extracted to a named constant (e.g., `MINIMUM-LIQUIDATION-BLOCK-GAP`) for clarity and single-point-of-change.
5. **Dead function parameter** (`liquidator-v1:503`): `ensure-non-zero-repay-amount` accepts `collateral-price` but never uses it. The function only checks `(> amount u0)`.
6. **Copy-paste bug in event log** (`withdrawal-caps-v1:271`): `set-debt-cap` logs the wrong old value. The print statement reads `old-value: (var-get lp-cap-factor)` instead of `old-value: (var-get debt-cap-factor)`. Compare with the correct `set-lp-cap` at line 258. Any off-chain system consuming this event will record the LP cap factor as the "old" debt cap value.

Recommendation

1. Rename `ERR_CONTRACT_NOT_ALLOWED` to `ERR-CONTRACT-NOT-ALLOWED` for consistency.
2. Remove the unused `ERR-LIQUIDATED-TOO-MUCH` constant.
3. Remove audit issue IDs from comments. Fix the time estimate or remove it:

```
-;; L-36: require at least 6 blocks (~1 min) between borrowing
   and liquidation
+;; require minimum block gap between borrowing and liquidation
```

4. And extract the magic number to a named constant:

```
+(define-constant MINIMUM-LIQUIDATION-BLOCK-GAP u6)
 ...
-(asserts! (>= stacks-block-height (+ (get borrowed-block
   position-for-block-check) u6)) ERR-LIQUIDATION-NOT-ALLOWED)
+(asserts! (>= stacks-block-height (+ (get borrowed-block
   position-for-block-check) MINIMUM-LIQUIDATION-BLOCK-GAP))
   ERR-LIQUIDATION-NOT-ALLOWED)
```

5. Remove the unused parameter

```
-(define-private (ensure-non-zero-repay-amount (amount uint)
   (collateral-price uint))
+(define-private (ensure-non-zero-repay-amount (amount uint))
```

Update call sites at line 128 and line 303.

Another solution would be to inline the check itself, since it is a single line in itself:

```
(asserts! (> amount u0) ERR-NON-ZERO-REPAY-AMOUNT)
```

6. Fix the copy-paste bug:

```
-(old-value: (var-get lp-cap-factor),
+(old-value: (var-get debt-cap-factor),
```

Resolution

Resolved, the recommended fix was implemented in **PR#62**.

[I-10] IR mul Function Uses Different Rounding Between Production and Utility Contracts

> **Severity:** *Informational* *(Resolved)*
>
> **Context:** *linear-kinked-ir-v1.clar linear-kinked-ir-utility.clar*

Description

The production interest rate contract uses floor division in its `mul` helper:

```
(define-private (mul (x uint) (y uint))
    (/ (* x y) one-12)
)
```

The utility contract uses banker's rounding (adds half before dividing):

```
(define-private (mul (x uint) (y uint))
    (/ (+ (* x y) (/ one-12 u2)) one-12)
)
```

The production contract systematically produces lower compounded interest than the utility. Off-chain systems (analytics, liquidation bots) using the utility contract see slightly higher projected interest than what actually accrues on-chain. The per-term difference is negligible (~1 unit at 12-decimal precision) but compounds through the 6 Taylor series terms used for exponential approximation.

Recommendation

Align the utility `mul` with the production version to ensure off-chain projections match on-chain behavior.

```
;; In linear-kinked-ir-utility.clar:
 (define-private (mul (x uint) (y uint))
-    (/ (+ (* x y) (/ one-12 u2)) one-12)
+    (/ (* x y) one-12)
 )
```

Resolution

Resolved, the recommended fix was implemented in **PR#62**.

[I-11] Exponent Upper Bound in Oracle Price Validation Is Too Permissive

> **Severity:** *Informational* *(Resolved)*
>
> **Context:** *pyth-adapter-v1.clar*

Description

The `pyth-adapter-v1` contract normalizes raw Pyth oracle prices to Granite's internal 8-decimal fixed-point format via the `convert-res` function. When the Pyth feed exponent is negative (the common case), the function safely adjusts decimals. However, when the exponent is zero or positive, it computes `price * pow(10, expo + PRICE_DECIMALS)`.

The exponent validation at line 133 allows exponents in the range `[-18, 18]`:

```
(asserts! (and (>= expo -18) (<= expo 18)) ERR-INVALID-EXPONENT)
```

The upper bound of 18 is too permissive. For `expo = 18`, `convert-res` evaluates `price * 10^26`, which overflows Clarity's 128-bit signed integer (max ~1.7 × 10^38) when `price > 1.7 × 10^12`. Since Clarity arithmetic overflow is a runtime abort (not a catchable error), this would DoS all price-dependent operations for that token.

However, the practical risk is very low for two reasons:

1. **No real Pyth feed uses positive exponents.** All standard Pyth price feeds (BTC, ETH, STX, USDC, etc.) use exponents in the range -8 to -5. The exponent encodes decimal precision, and positive values (representing prices in multiples of 10) are not used by any known feed. A positive exponent can only reach the contract if governance configures it via `update-price-feed-id`.
2. **The safe upper bound is 11.** Pyth's price field is an int64 (max ~9.2 × 10^18). The overflow condition is `pyth_max_int64 × 10^(expo + 8) > clarity_max_int128`, which gives `9.2 × 10^18 × 10^(expo+8) > 1.7 × 10^38`, solving to `expo > 11`. Any exponent ≤ 11 is safe against the full range of int64 prices, regardless of how large the raw price is. The current bound of 18 was likely chosen as a symmetric counterpart to -18 without accounting for the multiplication direction.

Recommendation

Tighten the upper bound from 18 to 11, which is the maximum safe value given Pyth's int64 price field and Clarity's int128 arithmetic:

```
;; In pyth-adapter-v1.clar, decode-pyth:
-(asserts! (and (>= expo -18) (<= expo 18)) ERR-INVALID-EXPONENT)
+(asserts! (and (>= expo -18) (<= expo 11)) ERR-INVALID-EXPONENT)
```

This preserves support for any theoretically valid positive exponent while eliminating the overflow window. No existing Pyth feed is affected since all use negative exponents.

Resolution

Resolved, the recommended fix was implemented in **PR#62**.

04 | Legal Disclaimer

This report is not, nor should be considered, an endorsement or disapproval of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any product or asset created by any team or project that contracts the consultant to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. The consultant's position is that each company and individual are responsible for their own due diligence and continuous security. The consultant's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology that is analyzed.

The assessment services provided by the consultant is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. Furthermore, because a single assessment can never be considered comprehensive, multiple independent assessments paired with a bug bounty program are always recommend.

For each finding, the consultant provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved, but they may not be tested or functional code. These recommendations are not exhaustive, and the clients are encouraged to consider them as a starting point for further discussion.

The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties. Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, the consultant does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

The consultant retains the right to re-use any and all knowledge and expertise gained during the audit process, including, but not limited to, vulnerabilities, bugs, or new attack vectors. The consultant is therefore allowed and expected to use this knowledge in subsequent audits and to inform any third party, who may or may not be our past or current clients, whose projects have similar vulnerabilities. The consultant is furthermore allowed to claim bug bounties from third-parties while doing so.